

Ludorivya.

Le reseau social de tes jeux video

Rapport de projet - SAE - PHP / PDO / MySQL

github.com/neorxprod/ludorivya

Ludorivya — guide complet du projet (pour le rapport et la soutenance)

Ce document explique **tout** : comment lancer le site, comment chaque morceau fonctionne, et pourquoi chaque choix a été fait. À lire avec [SCHEMA_RELATIONNEL.md](#) (la base) et le [README](#) (présentation).

1. Vue d'ensemble : comment le site fonctionne

Ludorivya est une application **PHP côté serveur** : il n'y a pas de framework JavaScript, pas d'API externe. Le cycle d'une page est toujours le même :

```

Navigateur —(requête HTTP)—> Apache —> PHP (page .php)
                                     | 1. bootstrap.php : session, CSRF, connexion
                                     | 2. requêtes SQL préparées vers MySQL
                                     | 3. génération du HTML avec les données
Navigateur <—(page HTML + CSS + JS)——
```

- **PHP** fait la logique et fabrique le HTML.
- **MySQL** stocke tout (jeux, comptes, avis, duels, forum...).
- **PDO** est le pont entre les deux : il envoie des **requêtes préparées** (la requête et les données voyagent séparément → injection SQL impossible).
- **JavaScript** (`assets/js/app.js`) ne fait que *dynamiser* : animations, validation des formulaires, et le mode versus en `fetch` (pour voter sans recharger la page).

2. Démarrer le site en local (XAMPP)

1. Ouvrir le **panneau XAMPP** → démarrer **Apache** et **MySQL** (boutons *Start*).
2. Importer la base : ouvrir `http://localhost/phpmyadmin` → onglet **Importer** → fichier `database/schema.sql` → **Exécuter**. (Ou en ligne de commande :
`C:\xampp\mysql\bin\mysql.exe -u root --default-character-set=utf8mb4 < database/schema.sql`.)
3. Rendre le projet visible par Apache : **copier le dossier du projet dans** `C:\xampp\htdocs\` et le nommer `ludorivya` (méthode la plus simple). *Alternative* (sans copier) : un raccourci de dossier, dans un terminal **administrateur** : `bat mklink /J C:\xampp\htdocs\ludorivya "C:\Users\rosyp\Documents\LUDORIVYA"`
4. Ouvrir **`http://localhost/ludorivya/public/`** — c'est l'URL officielle du site.

5. Compte de démo : `nora@example.test` / `Ludorivya2026!` .

⚠️ « **Le site n'est plus accessible alors qu'Apache est démarré** »

C'est presque sûrement parce que tu utilises **`http://localhost:8123`** : c'est un **serveur de test temporaire** (le serveur PHP intégré, lancé pendant les sessions de développement). Il s'arrête tout seul quand la session se termine — c'est normal.

👉 **L'URL pérenne, c'est celle d'Apache : `http://localhost/ludorivya/public/`**. Tant qu'Apache et MySQL sont démarrés dans XAMPP, elle marche toujours. (Si elle ne marche pas : vérifie que le `mklink` de l'étape 3 a bien été fait, et que MySQL est démarré — sinon le site affiche la page « Base de données non connectée ».)

3. L'arborescence, fichier par fichier

config/database.example.php	Modèle de connexion MySQL (hôte, utilisateur, mot de passe) Le vrai fichier config/database.php est IGNORÉ par Git (jamais de mot de passe dans un dépôt public).
src/Database.php	LA connexion PDO, centralisée. Options importantes : ERRMODE_EXCEPTION (les erreurs SQL lèvent des exceptions), EMULATE_PREPARES=false (vraies requêtes préparées MySQL).
src/bootstrap.php	Inclus en PREMIER par toutes les pages : - durcit le cookie de session (HttpOnly, SameSite), - démarre la session, génère le jeton CSRF, - se connecte à MySQL (ou prépare la page d'aide), - rafraîchit les infos de l'utilisateur connecté.
src/functions.php	La boîte à outils, ~20 fonctions : - e() : échappe le HTML (anti-XSS), utilisée sur TOUTE sort - csrf_field()/require_valid_csrf() : protection des formul - post_string/post_int/post_float : lecture VALIDÉE des for - validate_game_form() : validation complète d'un jeu ; - award_xp/level_from_xp/level_progress : le système XP ; - render_header/footer : le layout commun (navbar, flash, f - render_game_card/render_stars/render_pagination : composa
public/	LA RACINE WEB (seul dossier servi par Apache).
index.php	Accueil : héros 3D, marquee, étapes, à la une, promo versus
games.php	Catalogue : recherche LIKE, filtres genre/plateforme (EXIST
game.php	Fiche : jeu + studio (1-N), plateformes/genres (N-N), avis,
versus.php	bibliothèque, bilan de duels, jeux similaires (genres parta
duel_store.php	L'arène : le serveur tire 2 jeux au hasard et MÉMORISE la p
	en session (anti-triche), l'affichage maximise les jaquette
	Reçoit le vote en fetch : vérifie CSRF + paire servie + dél
	puis TRANSACTION { Elo gagnant/perdant + INSERT duel + XP }
	et renvoie la paire suivante en JSON. Gère aussi "je ne con
	pas" (action=skip : nouvelle paire, zéro écriture).
rankings.php	Classements : podium Elo, top notes (AVG+HAVING), joueurs (
forum.php / topic.php	Le forum : liste triée par dernière activité / le fil d'un
topic_store/reply_store/	Handlers POST du forum (création, réponse, suppressions
topic_delete/reply_delete	réservées à l'auteur via WHERE user_id).
create/edit + game_*.php	CRUD des jeux (transaction : jeu + liaisons N-N ensemble).
review_store/review_delete	Notes aux étoiles (1-10 → stockées sur 20), upsert via la
	contrainte UNIQUE (un avis par joueur et par jeu).
library_store/_delete	La bibliothèque (statut + heures, upsert pareil).
login/register/logout	Authentification (hash bcrypt, anti force brute, CSRF).
profile/users/stats	Profil (XP, niveau, mes jeux), joueurs publics, graphiques
public/assets/	
css/styles.css	TOUT le design : variables, effet flamme, boutons en biais,
	cartes 3D, arène versus, podium, forum, animations.

js/app.js	TOUTES les interactions : reveals au scroll, compteurs, til
vendor/	étoiles, particules, et le client du versus (fetch + animat
fonts/	Bootstrap + icônes EN LOCAL (le site marche sans internet).
img/covers/	La police Inter en local.
	Les 328 jaquettes (nommées par appid Steam).
database/	
schema.sql	TOUT : création de la base, 16 tables, données de démo.
dataset/games.json	Le dataset source des jeux (rédigé à la main).
dataset/fetch_covers.ps1	Téléchargement (une fois) des jaquettes depuis le CDN Steam
dataset/build_seed.php	Le générateur : JSON → schema.sql (+ simulation de 150 duel
.github/workflows/php.yml	CI GitHub : vérifie la syntaxe PHP à chaque push.

4. Une page type, pas à pas : le catalogue (games.php)

1. `require bootstrap.php` → session + `$pdo` connecté.
2. Lecture des filtres : `$_GET['q']` (tronquée à 100 caractères), `genre / platform` (validés `FILTER_VALIDATE_INT`), `page` (bornée).
3. Construction d'un `WHERE` dynamique **sans jamais concaténer de valeur** : les fragments SQL sont des constantes, les valeurs passent dans `$params` : `php $conditions[] = '(g.title LIKE :search_title OR ...)'`; `$params['search_title'] = '%' . $search . '%'`;
4. Le filtre par genre utilise la table N-N : `EXISTS (SELECT 1 FROM game_genres WHERE game_id = g.id AND genre_id = :genre_id)`.
5. Deux requêtes : un `COUNT(*)` (pour le nombre de pages) puis la page courante avec `LIMIT :limit OFFSET :offset` (liés en `PARAM_INT`).
6. Le HTML est généré ; chaque sortie passe par `e()`.

5. Le mode versus en détail (le cœur du projet)

Côté serveur (autorité totale) : 1. `versus.php` tire 2 jeux (`ORDER BY RAND() LIMIT 2`) et stocke leurs ids dans `$_SESSION['versus_pair']`. 2. Le client vote en `fetch POST` vers `duel_store.php` avec le jeton CSRF. 3. Le serveur vérifie : méthode POST → CSRF (`hash_equals`) → délai $\geq 1,5$ s depuis le dernier vote → **les ids votés sont exactement la paire servie**. Si un tricheur envoie d'autres ids : rejet 422. 4. Calcul **Elo** (le système des échecs) : `attendu = 1/(1+10^((eloB-eloA)/400))`, `delta = 32 × (1 - attendu)`. Battre un favori rapporte gros, battre un outsider rapporte peu. 5. **Transaction SQL** : `UPDATE` des deux Elo + `INSERT` du duel + `UPDATE` XP du votant — tout passe ou rien ne passe (`commit / rollback`). 6. Réponse JSON : deltas, nouveaux Elo, XP, niveau, série, et la **paire suivante** (mémorisée en session à son tour).

Côté client (que du visuel) : `app.js` anime le résultat (flash, secousse, confettis, compteurs Elo qui roulent) puis injecte la paire suivante sans recharger. Si JavaScript est coupé, le formulaire

classique fonctionne quand même (redirection + message).

« **Je ne connais pas** » : envoie `action=skip` → le serveur sert une nouvelle paire **sans aucune écriture** (l'Elo n'est pas pollué par des votes au hasard) et remet la série à zéro.

6. XP et niveaux

Action	XP
Vote versus	+5
Première note sur un jeu	+20
Jeu ajouté au catalogue	+30
Jeu ajouté à sa bibliothèque	+10
Sujet de forum créé	+15
Réponse au forum	+5

Le niveau n'est **pas stocké** : il est calculé (`niveau = 1 + xp ÷ 250`) — une seule source de vérité, pas de désynchronisation possible.

7. Les notes aux étoiles

- L'interface propose **10 étoiles** (survol animé, clic = choix), commentaire **facultatif**.
- La base stocke sur 20 (`rating = étoiles × 2`) — compatible avec les contraintes `CHECK` existantes.
- L'affichage reconvertit : `format_stars10()` (8,5/10) et `render_stars()` (★★★★★★★☆☆).
- La contrainte `UNIQUE (game_id, user_id) + INSERT ... ON DUPLICATE KEY UPDATE` garantissent **un avis par joueur et par jeu**, modifiable à volonté.

8. La sécurité (à réciter en soutenance)

Menace	Parade
Injection SQL	Requêtes préparées partout, <code>EMULATE_PREPARES=false</code>
XSS	<code>e()</code> (<code>htmlspecialchars</code>) sur toute sortie
CSRF	Jeton de session vérifié (<code>hash_equals</code>) sur tous les POST
Vol de session	Cookie HttpOnly + SameSite, <code>session_regenerate_id()</code> au login
Force brute	Pause de 60 s après 5 échecs de connexion
Triche au versus	Paire servie mémorisée côté serveur + délai entre votes
Open redirect	Redirections limitées aux pages internes (regex)
Donnée invalide	Validation serveur systématique (types, bornes, existence en base)
Fuite technique	Erreurs loguées (<code>error_log</code>), jamais affichées

La règle d'or : **le serveur ne fait jamais confiance au client**. Tout ce que le navigateur envoie est re-validé, recalculé, re-vérifié.

9. Le pipeline du catalogue (comment on a eu 328 vrais jeux)

1. `dataset/games.json` : un dataset rédigé jeu par jeu — titre réel, studio, date, PEGI, genres, plateformes, metaspcore approximatif, description française **originale** (rien n'est copié des fiches officielles), et l'**appid Steam**.
2. `dataset/fetch_covers.ps1` : pour chaque appid, télécharge la jaquette officielle 600×900 depuis le CDN public de Steam vers `public/assets/img/covers/`. Si l'appid est faux, il n'y a pas d'image → le jeu est écarté. **Le téléchargement sert donc de validation**.
3. `dataset/build_seed.php` : lit le JSON, déduplique les studios, mappe genres/plateformes, **simule 150 duels Elo** (graine aléatoire fixe → reproductible ; le jeu au meilleur metaspcore gagne 70 % du temps) et écrit `schema.sql` complet.

Avantage : la base se **régénère d'une commande**, et ajouter 100 jeux = ajouter des lignes au JSON puis relancer les deux scripts.

10. Git : comment le projet est versionné

- **Une branche par chantier** (`refonte-site` , `refonte-arena` ...), fusionnée dans `main` avec `--no-ff` (le merge reste visible dans l'historique).
- **Des commits par fonctionnalité** avec des messages en français qui disent *ce que* le commit apporte.

- `config/database.php` est gitignoré (jamais de secret dans le dépôt) ;
`database.example.php` sert de modèle.
- La **CI** (`.github/workflows/php.yml`) vérifie la syntaxe de tous les fichiers PHP à chaque push : un commit cassé est signalé par GitHub.

11. Mettre le site « vraiment en ligne » : ce qu'il faut savoir

GitHub ne peut pas héberger ce site. GitHub Pages ne sert que des fichiers **statiques** (HTML/CSS/JS) : il n'exécute **ni PHP ni MySQL**. Le dépôt GitHub héberge le *code*, pas le *serveur*.

Pour mettre le site en ligne il faut un hébergeur **PHP + MySQL** (la même stack qu'XAMPP, donc on reste 100 % dans le sujet). Deux options gratuites sérieuses :

1. **AlwaysData** (français, recommandé) — offre gratuite 100 Mo : - créer un compte sur alwaysdata.com → tu obtiens `toncompte.alwaysdata.net` ; - Bases de données → MySQL → créer une base + un utilisateur ; - importer `database/schema.sql` via leur phpMyAdmin ; - déposer les fichiers du projet par SFTP (FileZilla) ; - créer `config/database.php` sur le serveur avec les identifiants AlwaysData ; - Sites → faire pointer le site vers le dossier `public/`.
2. **InfinityFree** — gratuit aussi, même principe (mais publicité et limites plus agressives).

⚠ À vérifier avant : le poids des jaquettes (~7 Mo, OK pour 100 Mo) et le fait que **le sujet demande un serveur local XAMPP** — l'hébergement en ligne est un **bonus de démonstration**, pas une exigence. Mentionne-le comme tel dans le rapport.

12. Questions probables en soutenance (et les réponses)

- « *Pourquoi PDO plutôt que mysqli ?* » → API objet propre, requêtes préparées nommées, portabilité, et c'est l'exigence du sujet.
- « *Comment empêchez-vous la triche au versus ?* » → Le serveur choisit la paire, la mémorise en session, recalcule tout lui-même dans une transaction ; le client n'est qu'un écran.
- « *Que se passe-t-il si deux votes arrivent en même temps ?* » → Chaque vote est une transaction ; MySQL sérialise les UPDATE sur les mêmes lignes (verrous de ligne InnoDB).
- « *Pourquoi le niveau n'est-il pas une colonne ?* » → C'est une donnée **dérivée** de l'XP : la stocker créerait un risque d'incohérence (anomalie de mise à jour).
- « *Et si vous aviez 10 millions de jeux ?* » → Voir [BONUS_NOSQL.md](#) : index, dénormalisation, puis NoSQL pour la recherche et les compteurs chauds.